

LINQ Joins in C#

In this article, I will give you an overview of **LINQ Joins in C#**. Please read our previous article discussing the [LINQ ToLookup Method in C#](#) with Examples. One powerful feature of LINQ is its ability to perform joins, which allow you to combine data from two or more sources based on a common key or relation. As part of this article, we will discuss the following pointers.

1. **What are LINQ Join Operations?**
2. **Why do we need to perform the Join Operations?**
3. **What are the different methods available to perform Join Operations in LINQ?**
4. **Finally, what are the different types of LINQ Joins?**

If you have any experience in database systems like SQL Server, Oracle, MySQL, etc., then you may be familiar with SQL Joins. LINQ Joins are not different. They also merge data from two or more sources into a single result set based on some common property.

What are LINQ Joins?

According to Microsoft documentation, **joining two data sources involves combining objects in one data source with objects that share a common attribute in another data source.**

We can simplify the above definition as **Join operations are used to fetch the data from two or more data sources based on some common properties present in both data sources.**

Types of LINQ Join:

- **Inner Join (Join):** The Join method performs an inner join by correlating the elements of two collections based on matching keys. It returns a new collection that includes elements from both collections where the keys match. If an element from either collection does not have a match in the other, it is not included in the result.
- **Group Join (GroupJoin):** The GroupJoin method is similar to an inner join, but instead of returning a flat collection of matched pairs, it groups matching elements from the second collection for each element in the first collection. This operation is useful for hierarchical data relationships, such as when you want to associate each item in one collection with a collection of related items in another.
- **Left Outer Join:** While LINQ does not have a built-in method for left outer joins like it does for inner and group joins, you can achieve a left outer join by using the GroupJoin method followed by the SelectMany method with a DefaultIfEmpty method call. This operation returns all elements from the first collection and, for elements of the first collection that have matches in the second collection, includes the matching elements.
- **Cross Join:** In a cross join, every element of the first collection is combined with every element of the second collection, resulting in a Cartesian product of the two collections. This type of join doesn't require a key for joining and is implemented using the SelectMany method.

Why do we need to Perform Join Operations?

Let's understand why we need to perform Join Operations with an example. Let's say we have the following three data sources (Employee, Department, and Address).

Employee				Department		Address	
ID	Name	DeptID	AddressID	ID	DeptName	ID	AddressLine
1	Priyanka	10	101	10	IT	101	AddressLine1
2	Preety	20	102	20	HR	102	AddressLine2
3	Anurag	10	103			103	AddressLine3
4	Sambit	20	104			104	AddressLine4

Now, we need to fetch the data from the above three data sources, as shown below. Here, we are fetching ID and Name from the Employee table, DeptName from the Department table, and AddressLine from the Address table.

ID	Name	DeptName	AddressLine
1	Priyanka	IT	AddressLine1
2	Preety	HR	AddressLine2
3	Anurag	IT	AddressLine3
4	Sambit	HR	AddressLine4

As you can see, the above result set contains data from all three sources. The most important point you need to understand is that while performing join, you must have a common property in all the data sources involved. In this case, the common property between the Employee and the Department is the **Department ID**, which is present in both data sources. In the same line, **Address ID** is the common property between the Employee and Address data sources. So, in scenarios like this, we need to use LINQ Join to fetch the data.

What are the Methods Available in LINQ to Perform the Join Operations?

LINQ provides several methods for performing joins, similar to SQL joins, to combine data from two or more collections based on a common key or condition. The primary join methods in LINQ are Join, GroupJoin, and SelectMany. Here's how you can use them:

LINQ Join Method:

The Join method combines two sequences based on matching keys from each sequence. It returns a new sequence of combined elements.

Syntax:

```
var result = from item1 in collection1
join item2 in collection2 on item1.Key equals item2.Key
select new { Item1 = item1, Item2 = item2 };
```

The above query performs a join operation between two collections (collection1 and collection2) based on a common key (Key) that both elements in the collections have. It then projects (selects) a new anonymous type that includes each paired item from both collections. Here's a step-by-step breakdown:

- **from item1 in collection1:** This part of the query begins by iterating over each element in collection1. Each element in the iteration is referred to as item1.

- **join item2 in collection2 on item1.Key equals item2.Key:** This clause performs an inner join between collection1 and collection2. For each element (item1) in collection1, it looks for elements (item2) in collection2 where the Key property of item1 matches the Key property of item2. Only items that have a matching key in both collections are included in the result.
- **select new { Item1 = item1, Item2 = item2 }:** For each pair of matched items (item1 from collection1 and item2 from collection2), a new anonymous type is created. This new type has two properties: Item1, which refers to the item1 object, and Item2, which refers to the item2 object.

The result of this query, i.e., result, is a collection of the newly created anonymous types. Each element in this collection represents a pair of items from collection1 and collection2 that share the same Key value. This is particularly useful for correlating data from two separate sources based on a common identifier or key.

Example to Understand LINQ Join Method:

The following example code demonstrates how to perform an inner join between two collections, customers and orders, based on a common key (CustomerId in orders matching Id in customers). The aim is to pair each order with the name of the customer who made it. This example demonstrates how LINQ can be used to manipulate and access data from in-memory collections with SQL-like syntax for joining, filtering, and selecting.

Please explain the following Linq Query

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            var customers = new List<Customer>
            {
                new Customer { Id = 1, Name = "Alice" },
                new Customer { Id = 2, Name = "Bob" },
                new Customer { Id = 3, Name = "John" }
            };
            var orders = new List<Order>
            {
                new Order { OrderId = 101, CustomerId = 1 },
                new Order { OrderId = 102, CustomerId = 2 },
                new Order { OrderId = 103, CustomerId = 1 },
                new Order { OrderId = 104, CustomerId = 3 }
            };
            var result = from customer in customers
                        join order in orders on customer.Id equals order.CustomerId
                        select new { CustomerName = customer.Name, OrderId =
order.OrderId };
            foreach (var item in result)
            {
                Console.WriteLine($"Customer: {item.CustomerName}, Order ID:
{item.OrderId}");
            }
        }
    }
}
```

```

    }
    Console.ReadKey();
}
}
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
}
public class Order
{
    public int OrderId { get; set; }
    public int CustomerId { get; set; }
}
}

```

Code Explanation:

Initialization of Collections:

- Two lists are initialized – one containing customers (List<Customer>) and the other containing orders (List<Order>). Each Customer has an Id and a Name, while each Order has an OrderId and a CustomerId, where CustomerId links the order to a specific customer.

LINQ Query:

- The from keyword is used to start the query, indicating the source of the data (customers).
- The join operation is then performed, where each customer is joined with order in the orders list based on a matching condition (customer.Id equals order.CustomerId). This join operation finds all pairs of customers and orders where the customer's ID matches the customer ID specified in the order.
- The select clause specifies the shape of the resulting elements. In this case, it creates an anonymous type for each matching pair, containing CustomerName (from customer.Name) and OrderId (from order.OrderId).

Execution and Output:

- The query is executed when the foreach loop iterates over the result. LINQ queries are executed lazily, meaning the query isn't actually run until its results are enumerated (in this case, by the foreach loop).
- Inside the loop, it prints out the name of the customer and the order ID for each matching pair found by the query.

When you run the above code, you will get the following output:

```

Customer: Alice, Order ID: 101
Customer: Alice, Order ID: 103
Customer: Bob, Order ID: 102
Customer: John, Order ID: 104

```

LINQ GroupJoin Method:

The GroupJoin method is similar to Join, but it groups elements from the first sequence and associates them with matching elements from the second sequence.

Syntax:

```
var result = from item1 in collection1
join item2 in collection2 on item1.Key equals item2.Key into grouped
select new { Item1 = item1, GroupedItems = grouped };
```

The above LINQ query performs a group join between two collections, i.e., collection1 and collection2, based on a common key (Key) that both elements in the collections possess. It then projects (selects) a new anonymous type for each item in collection1 that includes the item itself and a collection of matched items from collection2. Here's a detailed explanation:

- **from item1 in collection1:** This starts the query by iterating over each element in collection1, referring to each element as item1.
- **join item2 in collection2 on item1.Key equals item2.Key into grouped:** This clause performs a group join between collection1 and collection2. For each element (item1) in collection1, it finds all matching elements (item2) in collection2 where item1.Key equals item2.Key. Instead of returning matched pairs directly, it groups all matching item2 elements for each item1 into a collection named grouped. If item1 does not have any matching elements in collection2, grouped will be an empty collection for that item1.
- **select new { Item1 = item1, GroupedItems = grouped }:** For each item1 in collection1, this creates a new anonymous type with two properties: Item1, which holds the item1 object itself, and GroupedItems, which holds the collection of matched item2 items from collection2 (the grouped collection).

The result of this query, i.e., result, is a collection of the anonymous types created by the select clause. Each element of this collection represents an item1 from collection1 and a collection of all corresponding item2 elements from collection2 that have the same key. This pattern is useful when you want to maintain a one-to-many relationship between elements in collection1 and elements in collection2, allowing each item1 to be associated with none, one, or many item2 elements based on the shared key.

Example to Understand LINQ GroupJoin Method:

The following example demonstrates a “group join” operation between two collections, departments and employees, based on a common key (DepartmentId). The purpose is to organize employees by their respective departments. The following LINQ query illustrates how to use group join to associate related data from two collections, allowing for a hierarchical representation of the data (departments containing their employees).

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
```

```

{
    var departments = new List<Department>
    {
        new Department { DepartmentId = 1, Name = "HR" },
        new Department { DepartmentId = 2, Name = "IT" }
    };
    var employees = new List<Employee>
    {
        new Employee { EmployeeId = 101, DepartmentId = 1, Name = "Ali" },
        new Employee { EmployeeId = 102, DepartmentId = 2, Name = "Bob" },
        new Employee { EmployeeId = 103, DepartmentId = 1, Name = "Charlie" },
        new Employee { EmployeeId = 104, DepartmentId = 2, Name = "John" },
        new Employee { EmployeeId = 105, DepartmentId = 1, Name = "Smith" }
    };
    //Performing Group Join
    var result = from department in departments
                 join employee in employees on department.DepartmentId
                 where employee.DepartmentId == department.DepartmentId
                 into grouped
                 select new { DepartmentName = department.Name, Employees = grouped };
    foreach (var item in result)
    {
        Console.WriteLine($"Department: {item.DepartmentName}");
        foreach (var employee in item.Employees)
        {
            Console.WriteLine($"    \tEmployee: {employee.Name}");
        }
        Console.WriteLine();
    }
    Console.ReadKey();
}

public class Department
{
    public int DepartmentId { get; set; }
    public string Name { get; set; }
}

public class Employee
{
    public int EmployeeId { get; set; }
    public int DepartmentId { get; set; }
    public string Name { get; set; }
}
}

```

Code Explanation:

Initialization of Collections: Two lists are initialized:

- **departments:** Contains instances of the Department class, each with a DepartmentId and a Name.

- **employees:** Contains instances of the Employee class, each with an EmployeeId, DepartmentId, and Name. The DepartmentId field is used to associate an employee with a department.

LINQ Query – Group Join:

- The from keyword specifies the primary collection to start with (departments in this case).
- The join ... on ... equals ... into syntax performs the group join. It attempts to match each department from departments with any employee from employees where department.DepartmentId equals employee.DepartmentId. Instead of producing a flat result set like a regular join, the into grouped part groups all matching employees for each department into a single collection (grouped).
- The select clause then creates a new anonymous type for each department, including DepartmentName (the name of the department) and Employees (the collection of employees that belong to this department, as grouped in the previous step).

Execution and Output:

- The query is executed lazily when the foreach loop iterates over result. At this point, the group join operation is carried out, and the results are materialized.
- The outer foreach loop iterates over each department in the result. It prints the name of the department.
- The inner foreach loop iterates over the Employees collection of each department (item.Employees), printing the name of each employee belonging to that department.

When you run the above code, you will get the following output:

```
Department: HR
    Employee: Alice
    Employee: Charlie
    Employee: Smith

Department: IT
    Employee: Bob
    Employee: John
```

SelectMany Method:

The SelectMany method is used to perform multiple joins or flatten nested collections.

Syntax:

```
var result = from item1 in collection1
from item2 in item1.SubCollection
select new { Item1 = item1, Item2 = item2 };
```

This LINQ query demonstrates a select operation that involves a parent collection (collection1) and a child collection (SubCollection within each item of collection1). Here's the breakdown:

- **from item1 in collection1:** This initiates the query by iterating over each element in collection1, where each element is referred to as item1.
- **from item2 in item1.SubCollection:** For each item1, this part iterates over its SubCollection. This is essentially a nested loop, where for each item in the outer loop (item1), the inner loop iterates over each element in item1's SubCollection, referred to here as item2.
- **select new { Item1 = item1, Item2 = item2 }:** For each iteration of the inner loop (for each item2 in each item1.SubCollection), a new anonymous type is created. This new type contains two properties:
 - **Item1**, which holds the reference to the current item1 from the outer loop.
 - **Item2**, which holds the reference to the current item2 from the inner loop (an element from the SubCollection of item1).

The result, i.e., result, is a collection of the new anonymous types created by the select clause. Each element of this collection represents a combination of an item from collection1 and an item from that item's SubCollection. This query is useful when you need to flatten a hierarchical structure (such as parent-child relationships) into a flat collection of pairs or when you need to perform operations on each combination of parent and child elements in a collection.

Example to Understand LINQ SelectMany Method:

The following example demonstrates how to perform a “select many” operation, which flattens a collection of collections. Here, it flattens the list of courses for each student into a single collection of student-course pairs. This is useful when you need to work with a nested collection as if it were a single list.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            var students = new List<Student>
            {
                new Student { Id = 1, Name = "Alice", Courses = new List<string>
                { "ASP.NET Core" } },
                new Student { Id = 2, Name = "Bob", Courses = new List<string>
                { "SQL Server" } },
                new Student { Id = 3, Name = "John", Courses = new List<string>
                { "PHP" } }
            };
            var result = from student in students
                        from course in student.Courses
                        select new { StudentName = student.Name, CourseName =
                        course };
            foreach (var item in result)
            {
                Console.WriteLine($"Student: {item.StudentName}, Course:
                {item.CourseName}");
            }

            Console.ReadKey();
        }
    }
}
```

```

    }
}
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public List<string> Courses { get; set; }
}
}

```

Code Explanation:

Initialization of Collections: A list of Student objects is initialized. Each Student has an Id, a Name, and a list of Courses they are enrolled in.

LINQ Query:

- The first from clause (**from student in students**) iterates over each student in the students collection.
- The second from clause (**from course in student.Courses**) iterates over each course in the current student's Courses list. This is where the "select many" operation happens. Instead of dealing with a list of courses for each student, this operation allows the query to work with each course individually, effectively flattening the structure.
- The select clause creates a new anonymous type for each student-course pair, containing StudentName (from student.Name) and CourseName (from course, which represents each course the student is enrolled in).

Execution and Output:

- The query is executed lazily when the foreach loop iterates over the result. At this point, the "select many" operation is carried out, and the results are materialized.
- The loop prints out the name of each student along with each course they are enrolled in. This results in a flat list where each student-course combination is treated as a separate entry.

When you run the above code, you will get the following output:

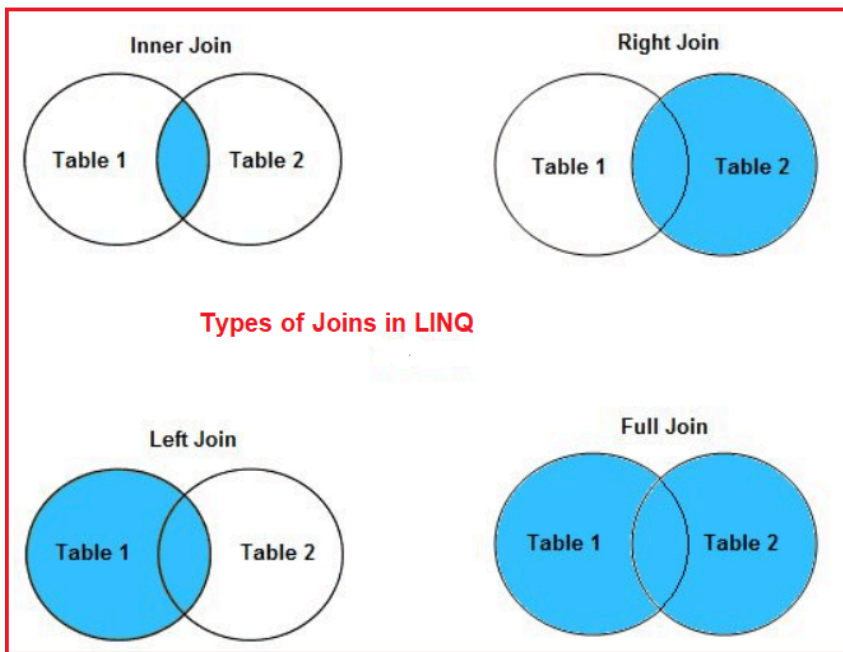
```

Student: Alice, Course: C#
Student: Alice, Course: ASP.NET Core
Student: Bob, Course: MySQL
Student: Bob, Course: SQL Server
Student: John, Course: Java
Student: John, Course: PHP

```

What are the Different Types of LINQ Joins Available in C#?

In LINQ, we can perform different types of joins, such as Inner, Left, Right, Full, and Cross Join. Please have a look at the image below for a better understanding.



Inner Join: The Inner Join returns only the matching rows from both the data sources involved in the join by removing the non-matching records.

Left Outer Join: The LEFT OUTER JOIN retrieves all the matching rows from both the data sources involved in the join and non-matching rows from the left-side data source. In this case, the unmatched data will take a null value.

Right Outer Join: The RIGHT OUTER JOIN retrieves all the matching rows from the data sources involved in the join and non-matching rows from the right-side data source. In this case, the unmatched data will take NULL values.

Full Outer Join: The Full Outer Join retrieves all the matching and non-matching rows from both data sources involved in the JOIN. The non-matching data in such cases will take the NULL values.

Cross Join: In Cross Join, each record of a data source is joined with each record of the other data source.

In the next article, I will discuss the [LINQ Inner Join in C#](#) with Examples. In this article, I try to give an overview of **LINQ Join in C#**, and I hope you enjoy this article.